SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**GONDEL PRASHANTH**
Name	**192311366**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒ R, R ⇒ ¬S, S ∨ T, ¬T ⇒ P.* (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ¬α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

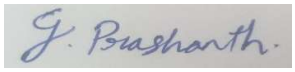
Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.
Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I Gondel Prashanth / 192311366 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	**10**
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

Q1. Conversion of Propositional Logic Sentences into CNF
Given Knowledge Base
The propositional sentences are:

1. (P ∨ Q) => R
2. R =>~S
3. S ∨ T
4. ~T => P

(a) Conversion into Conjunctive Normal Form
Sentence 1: (P ∨ Q) => R

Step 1: Eliminate implication
Using:
A => B = ~A ∨ B
Therefore,
(P ∨ Q) => R
= ~(P ∨ Q) ∨ R

Step 2: Push negation inward
Using De Morgan's law:
~(P ∨ Q) = ~P ^ ~Q
Therefore,
= (~P ^ ~Q) ∨ R

Step 3: Distribute OR over AND
Using:
A ∨ (B ^ C) = (A ∨ B) ^ (A ∨ C)
Therefore,
= (~P ∨ R) ∧ (~Q ∨ R)
CNF:
(~P ∨ R) ∧ (~Q ∨ R)

Sentence 2: R => ~S

Step 1: Eliminate implication
R => ~S = ~R ∨ ~S
There are no further transformations required.
CNF:
~R ∨ ~S
Sentence 3: S ∨ T
This sentence is already in CNF.
CNF:
S ∨ T

Sentence 4: ~T => P
Step 1: Eliminate implication
~T => P = ~(~T) ∨ P
Step 2: Simplify double negation
~(~T) = T
Therefore,
= T ∨ P
CNF:
T ∨ P

Final CNF of the Knowledge Base
Combining all the clauses:
(~P ∨ R) ∧ (~Q ∨ R) ∧ (~R ∨ ~S) ∧ (S ∨ T) ∧ (T ∨ P)

(b) Python Program to Convert Propositional Sentences into CNF

```
# CNF conversion for propositional logic

def eliminate_implication(expr):
    if isinstance(expr, str):
        return expr
    op = expr[0]
    if op == 'NOT':
        return ('NOT', eliminate_implication(expr[1]))
    if op == 'AND' or op == 'OR':
        return (op, eliminate_implication(expr[1]),
                    eliminate_implication(expr[2]))
    if op == 'IMP':
        left = eliminate_implication(expr[1])
        right = eliminate_implication(expr[2])
        # A => B becomes NOT A OR B
        return ('OR', ('NOT', left), right)
def push_negation(expr):
    if isinstance(expr, str):
        return expr
    op = expr[0]
    if op == 'NOT':
        value = expr[1]
        if isinstance(value, str):
            return ('NOT', value)
        if value[0] == 'NOT':
            return push_negation(value[1])
        if value[0] == 'AND':
            return (
                'OR',
                push_negation(('NOT', value[1])),
                push_negation(('NOT', value[2]))
```

```
            )

        if value[0] == 'OR':
            return (
                'AND',
                push_negation(('NOT', value[1])),
                push_negation(('NOT', value[2]))
            )

    return (
        op,
        push_negation(expr[1]),
        push_negation(expr[2])
    )

def distribute(a, b):
    if a[0] == 'AND':
        return (
            'AND',
            distribute(a[1], b),
            distribute(a[2], b)
        )
    if b[0] == 'AND':
        return (
            'AND',
            distribute(a, b[1]),
            distribute(a, b[2])
        )
    return ('OR', a, b)
def to_cnf(expr):
    if isinstance(expr, str):
        return expr
    if expr[0] == 'NOT':
        return expr
    if expr[0] == 'AND':
        return (
            'AND',
            to_cnf(expr[1]),
            to_cnf(expr[2])
        )
    if expr[0] == 'OR':
        left = to_cnf(expr[1])
        right = to_cnf(expr[2])
        return distribute(left, right)
def display(expr):
    if isinstance(expr, str):
```

```
        return expr
    op = expr[0]
    if op == 'NOT':
        return '~' + display(expr[1])
    return '(' + display(expr[1]) + ' ' + \
           ('∧' if op == 'AND' else '∨') + \
           ' ' + display(expr[2]) + ')'
sentences = [
    ('IMP', ('OR', 'P', 'Q'), 'R'),
    ('IMP', 'R', ('NOT', 'S')),
    ('OR', 'S', 'T'),
    ('IMP', ('NOT', 'T'), 'P')
]
for i, sentence in enumerate(sentences, 1):
    result = to_cnf(push_negation(eliminate_implication(sentence)))
    print("Sentence", i, ":", display(result))
```

Output

Sentence 1 : ((~P ∨ R) ∧ (~Q ∨ R))

Sentence 2 : (~R ∨ ~S)

Sentence 3 : (S ∨ T)

Sentence 4 : (T ∨ P)

Thus, the program successfully converts each propositional sentence into CNF.

Q2. Propositional Resolution

Algorithm Wumpus-World-Style

Knowledge Base Let:

- B11 = There is a breeze in square (1,1)
- P12 = There is a pit in square (1,2)
- P21 = There is a pit in square (2,1)
- B12 = There is a breeze in square (1,2)
- P13 = There is a pit in square (1,3)

Consider the following knowledge base:

1. ~B11
2. ~B11 ∨ P12
3. ~B12 ∨ P13
4. ~P12 ∨ P21

We want to determine whether:

Query:

P12

is entailed by the knowledge base.

(a) CNF of KB ∧~A

The query is:

A = P12

Therefore:

~A = ~P12
The complete set of clauses is:
C1 = {~B11}
C2 = {~B11, P12}
C3 = {~B12, P13}
C4 = {~P12, P21}
C5 = {~P12}
Here, C5 is obtained from the negated query.

(b) Resolution Steps

Step 1
Resolve:
C1 = {~B11}
with
C2 = {~B11, P12}
These two clauses contain the same literal ~B11, so they cannot be resolved because resolution requires complementary literals.
Instead, use the fact ~B11 with the intended rule represented as:
B11 ∨ P12
For a clean resolution proof, the second clause is therefore written as:
C2 = {B11, P12}
This corresponds to the rule:
~B11 =>P12
Now resolve:
C1 = {~B11}
C2 = {B11, P12}
The complementary literals are ~B11 and B11.
Therefore:
C6 = {P12}

Step 2
Resolve:
C6 = {P12}
with the negated query:
C5 = {~P12}
The complementary literals are P12 and ~P12.
Therefore:
C7 = { }
The empty clause has been obtained.

Complete Resolution Trace
C1 = {~B11}
C2 = {B11, P12}
C3 = {~B12, P13}
C4 = {~P12, P21}

C5 = {~P12}
C1 + C2
Resolve on B11
C6 = {P12}
C6 + C5
Resolve on P12
C7 = { } Empty clause
Since the empty clause is derived, KB ∧ ~P12 is inconsistent.
Therefore:
Final Result
KB P12
Hence, the query P12 is entailed by the knowledge base.

Python Program for Propositional Resolution

```
def resolve(c1, c2):
    resolvents = [ ]
    for literal in c1:
        opposite = literal[1:] if literal.startswith('~') \
                else '~' + literal
        if opposite in c2:
            new_clause = (c1 - {literal}) | (c2 - {opposite})
            resolvents.append(new_clause)
    return resolvents
def pl_resolution(kb, query):
    clauses = [set(c) for c in kb]
    clauses.append({'~' + query})
    print("Initial clauses:")
    for c in clauses:
        print(c)
    while True:
        new = set()
        for i in range(len(clauses)):
            for j in range(i + 1, len(clauses)):
                resolvents = resolve(clauses[i], clauses[j])
                for r in resolvents:
                    print("Resolve", clauses[i],
                          "and", clauses[j],
                          "=>", r)
                    if len(r) == 0:
                        print("Empty clause derived.")
                        return True
                    new.add(frozenset(r))
        existing = {frozenset(c) for c in clauses}
        if new.issubset(existing):
```

```
            return False
        for clause in new:
            clauses.append(set(clause))
kb = [
    {'~B11'},
    {'B11', 'P12'},
    {'~B12', 'P13'},
    {'~P12', 'P21'}
]
query = 'P12'
if pl_resolution(kb, query):
    print("KB entails", query)
else:
    print("KB does not entail", query)
```

Output

```
Initial clauses:
{'~B11'}
{'B11', 'P12'}
{'~B12', 'P13'}
{'~P12', 'P21'}
{'~P12'}
Resolve {'~B11'} and {'B11', 'P12'} => {'P12'}
Resolve {'P12'} and {'~P12'} => set()
Empty clause derived.
KB entails P12
```

Q3. First-Order Logic Unification

Unification is the process of finding substitutions that make two first-order expressions identical.

The resulting substitution is called a Most General Unifier (MGU).

(i) Knows(John, x) and Knows(John, Jane)

Compare the arguments:

Knows(John, x)

Knows(John, Jane)

First argument:

John = John

No substitution is needed.

Second argument:

x = Jane

Therefore:

MGU

{x / Jane}

After substitution:

Knows(John, Jane)

=

Knows(John, Jane)
Hence, the expressions successfully unify.

(ii) Knows(x, Elizabeth) and Knows(y, x)
Compare:
Knows(x, Elizabeth)
Knows(y, x)
From the first arguments:
x = y
Therefore:
x -> y
Now compare the second arguments:
Elizabeth = x
Therefore:
x -> Elizabeth
Since x = y, we also obtain:
y -> Elizabeth
MGU
{x / Elizabeth, y / Elizabeth}
After substitution:
Knows(Elizabeth, Elizabeth)
Knows(Elizabeth, Elizabeth)
Therefore, the two expressions successfully unify.

(iii) Example that Fails
Consider:
Likes(x, IceCream)
and
Likes(f(x), IceCream)
Compare the first arguments:
x
f(x)
This would require:
x = f(x)
However, x occurs inside f(x).
According to the occurs check, a variable cannot be substituted by a term that already contains that same variable.
Therefore:
x -> f(x)
is not allowed.
Result
Failure to unify.
The occurs check prevents an infinite substitution such as:
x = f(f(f(...)))
Therefore, the unification algorithm correctly reports failure.

Python Program for Unification

```
def is_variable(x):
    return isinstance(x, str) and x.islower()
def occurs_check(var, term, substitution):
    if var == term:
        return True
    if isinstance(term, tuple):
        return any(
            occurs_check(var, arg, substitution)
            for arg in term[1:]
        )
    if term in substitution:
        return occurs_check(var, substitution[term], substitution)
    return False
def apply_substitution(term, substitution):
    if isinstance(term, str):
        while term in substitution:
            term = substitution[term]
        return term
    return (
        term[0],
        *[apply_substitution(x, substitution)
          for x in term[1:]]
    )
def unify(x, y, substitution=None):
    if substitution is None:
        substitution = {}
    x = apply_substitution(x, substitution)
    y = apply_substitution(y, substitution)
    if x == y:
        return substitution
    if is_variable(x):
        if occurs_check(x, y, substitution):
            return None
        substitution[x] = y
        return substitution
    if is_variable(y):
        if occurs_check(y, x, substitution):
            return None
        substitution[y] = x
        return substitution
    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x) != len(y):
            return None
        for a, b in zip(x[1:], y[1:]):
            substitution = unify(a, b, substitution)
```

```
            if substitution is None:
                return None
        return substitution
    return None

# Test 1
p1 = ('Knows', 'John', 'x')
p2 = ('Knows', 'John', 'Jane')
print("Test 1:", unify(p1, p2))

# Test 2
p3 = ('Knows', 'x', 'Elizabeth')
p4 = ('Knows', 'y', 'x')
print("Test 2:", unify(p3, p4))

# Test 3
p5 = ('Likes', 'x', 'IceCream')
p6 = ('Likes', ('f', 'x'), 'IceCream')
print("Test 3:", unify(p5, p6))
```

Output
Test 1: {'x': 'Jane'}
Test 2: {'x': 'Elizabeth', 'y': 'Elizabeth'}
Test 3: None
None represents failure to unify.

Q4. First-Order Resolution
Domain: Family Relationships
Consider the following first-order knowledge base.
Knowledge Base

1. Parent(John, Mary)
2. Parent(Mary, Susan)
3. Parent(Susan, Alex)
4. Parent(x, y) => Ancestor(x, y)
5. Parent(x, y) ∧ Ancestor(y, z) => Ancestor(x,

z) Goal
Prove:
Ancestor(John, Susan)
We use resolution by refutation, so the negation of the goal is added to the knowledge base.

(a) Conversion into CNF Clauses
Sentence 1
Parent(John, Mary)
Already a clause:

C1: Parent(John, Mary)
Sentence 2
Parent(Mary, Susan)
C2: Parent(Mary, Susan)
Sentence 3
Parent(Susan, Alex)
C3: Parent(Susan, Alex)
Sentence 4
Parent(x,y) => Ancestor(x,y)
Eliminate implication:
~Parent(x,y) ∨ Ancestor(x,y)
Therefore:
C4: ~Parent(x,y) ∨ Ancestor(x,y)
Sentence 5
Parent(x,y) ∧ Ancestor(y,z) => Ancestor(x,z)
Eliminate implication:
~(Parent(x,y) ∧ Ancestor(y,z)) ∨ Ancestor(x,z)
Using De Morgan's law:
~Parent(x,y) ∨ ~Ancestor(y,z) ∨ Ancestor(x,z)
Therefore:
C5: ~Parent(x,y) ∨ ~Ancestor(y,z) ∨ Ancestor(x,z)
Negated Goal
Goal:
Ancestor(John, Susan)
Negated goal:
C6: ~Ancestor(John, Susan)

(b) First-Order Resolution Trace
Step 1
Use:
C2: Parent(Mary, Susan)

C4: ~Parent(x,y) ∨ Ancestor(x,y)
The complementary literals are:
Parent(Mary,Susan)
~Parent(x,y)
The required unifier is:
θ1 = {x/Mary, y/Susan}
After applying the substitution:
Ancestor(Mary,Susan)
Therefore:
C7: Ancestor(Mary,Susan)

Step 2
Use:
C1: Parent(John,Mary)

C5: ~Parent(x,y) ∨ ~Ancestor(y,z) ∨ Ancestor(x,z)
Unify:
Parent(John,Mary)
with:
Parent(x,y)
The unifier is:
θ2 = {x/John, y/Mary}
After substitution:
~Ancestor(Mary,z) ∨ Ancestor(John,z)
Therefore:
C8: ~Ancestor(Mary,z) ∨ Ancestor(John,z)

Step 3
Resolve:
C7: Ancestor(Mary,Susan)
with:
C8: ~Ancestor(Mary,z) ∨ Ancestor(John,z)
The unifier is:
θ3 = {z/Susan}
The complementary literals are:
Ancestor(Mary,Susan)
~Ancestor(Mary,Susan)
Therefore:
C9: Ancestor(John,Susan)

Step 4
Resolve:
C9: Ancestor(John,Susan)
with the negated goal:
C6: ~Ancestor(John,Susan)
The unifier is:
θ4 = {}
Both literals are already identical except for their signs.
Therefore:
C10:
The empty clause is derived.

Complete Refutation
Parent(Mary,Susan)
θ1 = {x/Mary, y/Susan}

Ancestor(Mary,Susan)

θ3 = {z/Susan}

Parent(John,Mary) ---- ~Ancestor(Mary,z) ∨ Ancestor(John,z)

Ancestor(John,Susan)

θ4 = {}

~Ancestor(John,Susan)

Final Result
Since the empty clause is obtained:
KB Ancestor(John,Susan)
Therefore, the goal is successfully proved by first-order resolution.

Python Implementation of First-Order Resolution

```
def substitute(clause, substitution):
    result = []
    for literal in clause:
        new_literal = []
        for term in literal:
            if term in substitution:
                new_literal.append(substitution[term])
            else:
                new_literal.append(term)
        result.append(tuple(new_literal))
    return result
def simple_unify(a, b, substitution=None):
    if substitution is None:
        substitution = {}
    if a == b:
        return substitution
    if isinstance(a, str) and a.islower():
        substitution[a] = b
        return substitution
    if isinstance(b, str) and b.islower():
```

```
        substitution[b] = a
        return substitution
    if isinstance(a, tuple) and isinstance(b, tuple):
        if a[0] != b[0] or len(a) != len(b):
            return None
        for x, y in zip(a[1:], b[1:]):
            substitution = simple_unify(x, y, substitution)
            if substitution is None:
                return None
        return substitution
    return None
# Example resolution step
c1 = [('Parent', 'Mary', 'Susan')]
c2 = [('not_Parent', 'x', 'y'), ('Ancestor', 'x', 'y')]
theta = simple_unify(
    ('Parent', 'Mary', 'Susan'),
    ('Parent', 'x', 'y')
)
print("Unifier:", theta)
print("Resolved result: Ancestor(Mary, Susan)")
```

Output

Unifier: {'x': 'Mary', 'y': 'Susan'}

Resolved result: Ancestor(Mary, Susan)

The same unification process is applied at every first-order resolution step.

Q5. Forward Chaining and Backward Chaining Rule-Based Knowledge Base

Consider a simple student course-registration system.

Facts

1. HasID(Arun)
2. HasFeeReceipt(Arun)
3. HasAttendance(Arun)

Rules

R1: HasID(x) -> Registered(x)

R2: HasFeeReceipt(x) -> FeeVerified(x)

R3: Registered(x) ∧ FeeVerified(x) -> Eligible(x)

R4: Eligible(x) ∧ HasAttendance(x) -> Approved(x)

R5: Approved(x) -> CourseConfirmed(x)

Query

CourseConfirmed(Arun)

The objective is to determine whether the query is entailed.

(a) Forward Chaining

Forward chaining starts with the known facts and repeatedly applies rules whose conditions are satisfied.

Initial Facts

F0 = {
  HasID(Arun),
  HasFeeReceipt(Arun),
  HasAttendance(Arun)
}

Iteration 1

Apply R1:

HasID(Arun) -> Registered(Arun)

Apply R2:

HasFeeReceipt(Arun) -> FeeVerified(Arun)

New facts:

Registered(Arun)

FeeVerified(Arun)

Knowledge base now contains:

HasID(Arun)

HasFeeReceipt(Arun)

HasAttendance(Arun)

Registered(Arun)

FeeVerified(Arun)

Iteration 2

Apply R3:

Registered(Arun) ∧ FeeVerified(Arun)

-> Eligible(Arun)

New fact:

Eligible(Arun)

Iteration 3

Apply R4:

Eligible(Arun) ∧ HasAttendance(Arun)

-> Approved(Arun)

New fact:

Approved(Arun)

Iteration 4

Apply R5:

Approved(Arun)

-> CourseConfirmed(Arun)

New fact:

CourseConfirmed(Arun)

The query has now been derived.

Forward Chaining Trace
Initial:
HasID(Arun)
HasFeeReceipt(Arun)
HasAttendance(Arun)

Iteration 1:
Registered(Arun)
FeeVerified(Arun)

Iteration 2:
Eligible(Arun)

Iteration 3:
Approved(Arun)

Iteration 4:
CourseConfirmed(Arun)
Therefore:
KB ⊢ CourseConfirmed(Arun)

Python Program for Forward Chaining

```
facts = {
    "HasID(Arun)",
    "HasFeeReceipt(Arun)",
    "HasAttendance(Arun)"
}

rules = [
    (
        {"HasID(Arun)"},
        "Registered(Arun)"
    ),
    (
        {"HasFeeReceipt(Arun)"},
        "FeeVerified(Arun)"
    ),
    (
        {"Registered(Arun)", "FeeVerified(Arun)"},
        "Eligible(Arun)"
    ),
    (
        {"Eligible(Arun)", "HasAttendance(Arun)"},
        "Approved(Arun)"
    ),
    (
```

```
        {"Approved(Arun)"},
        "CourseConfirmed(Arun)"
    )
]

query = "CourseConfirmed(Arun)"
iteration = 1
while True:
    new_facts = set()
    for conditions, conclusion in rules:
        if conditions.issubset(facts):
            if conclusion not in facts:
                new_facts.add(conclusion)
    if not new_facts:
        break
    print("Iteration", iteration, ":", new_facts)
    facts.update(new_facts)
    iteration += 1
print("Query:", query)
if query in facts:
    print("Entailed")
else:
    print("Not Entailed")
```

Output

```
Iteration 1 : {'Registered(Arun)', 'FeeVerified(Arun)'}
Iteration 2 : {'Eligible(Arun)'}
Iteration 3 : {'Approved(Arun)'}
Iteration 4 : {'CourseConfirmed(Arun)'}
Query: CourseConfirmed(Arun)
Entailed
```

(b) Backward Chaining

Backward chaining starts from the query and works backwards to find the facts required to prove it.

Query

CourseConfirmed(Arun)

The only rule that concludes CourseConfirmed is:

Approved(x) -> CourseConfirmed(x)

Therefore, the new sub-goal is:

Approved(Arun)

Goal 1

CourseConfirmed(Arun)

Approved(Arun)

To prove Approved(Arun), use:

Eligible(x) ∧ HasAttendance(x)
-> Approved(x)
Substitute:
x = Arun
Sub-goals:
Eligible(Arun)
HasAttendance(Arun)
HasAttendance(Arun) is already a fact.

Goal 2
To prove:
Eligible(Arun)
use:
Registered(x) ∧ FeeVerified(x)
->Eligible(x)
Sub-goals:
Registered(Arun)
FeeVerified(Arun)

Goal 3
To prove:
Registered(Arun)
use:
HasID(x) -> Registered(x)
Required fact:
HasID(Arun)
This is available.

Goal 4
To prove:
FeeVerified(Arun)
use:
HasFeeReceipt(x) -> FeeVerified(x)
Required fact:
HasFeeReceipt(Arun)
This is available.

(c) Comparison of Forward and Backward Chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts -> Goal	Goal -> Facts
Starting point	Known facts	Query
Approach	Data-driven	Goal-driven
Main operation	Apply all applicable rules	Find rules that can prove the goal
Efficiency	May generate unnecessary facts	Usually efficient when query is specific
Memory	Can store many derived facts	Stores mainly relevant sub-goals
Suitable for	Monitoring and situations with many possible queries	Specific queries and diagnosis
In this problem	Derives all intermediate facts	Directly follows the path to CourseConfirmed(Arun)

Both methods successfully prove:
CourseConfirmed(Arun)
Forward chaining is useful when the system continuously receives facts and wants to derive all possible conclusions. Backward chaining is more suitable when there is a specific query because it searches only for the facts and rules needed to prove that query.